1) 数据集 & 拆分

- `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`
- `KFold(n_splits=5, shuffle=True, random_state=42)`
- `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`
- `TimeSeriesSplit(n_splits=5)`
- `cross_val_score(estimator, X, y, cv=5, scoring='roc_auc', n_jobs=-1)`
- `GridSearchCV(estimator, param_grid, cv=5, scoring='roc_auc', n_jobs=-1)`
- `RandomizedSearchCV(estimator, param_distributions, n_iter=50, cv=5, n_jobs=-1)`

```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score, GridSearchCV
```

2) 预处理（Preprocessing）

- 标准化/缩放
  - `StandardScaler()`, `MinMaxScaler()`, `RobustScaler()`, `PowerTransformer()`
- 编码
  - `OneHotEncoder(handle_unknown='ignore', sparse_output=True)`
- 缺失值
  - `SimpleImputer(strategy='median'|'most_frequent'|'constant')`, `KNNImputer(n_neighbors=5)`

- 特征加工
  - `PolynomialFeatures(degree=2, include_bias=False)`
  - `QuantileTransformer(output_distribution='normal')`
- 列级组合
  - `ColumnTransformer(transformers=[('num', scaler, num_cols), ('cat', ohe, cat_cols)], remainder='drop')`
- 流水线
  - `Pipeline(steps=[('preprocess', pre), ('model', clf)])`
  - `make_pipeline(StandardScaler(), LogisticRegression())`

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
```

---

3) 常见模型（Estimators）

3.1 线性 / 广义线性

- 分类：`LogisticRegression(max_iter=1000, class_weight='balanced')`
- 回归：`LinearRegression()`，`Ridge(alpha=1.0)`，`Lasso(alpha=0.001)`，`ElasticNet(alpha=0.001, l1_ratio=0.5)`
- 大样本线性：`SGDClassifier(loss='log_loss', max_iter=1000)`，`SGDRegressor(max_iter=1000)`

```
from sklearn.linear_model import LogisticRegression, LinearRegression, Ridge, Lasso, ElasticNet, SGDClassifier, SGDRegressor
```

3.2 树与集成

- 单树：`DecisionTreeClassifier()`，`DecisionTreeRegressor()`
- 随机森林：`RandomForestClassifier(n_estimators=300, n_jobs=-1)`，`RandomForestRegressor(...)`
- Boosting：
  - `GradientBoostingClassifier()`，`GradientBoostingRegressor()`
  - **推荐**：`HistGradientBoostingClassifier()`，`HistGradientBoostingRegressor()`
- ExtraTrees：`ExtraTreesClassifier()`，`ExtraTreesRegressor()`

```
from sklearn.tree import DecisionTreeClassifier,
DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier,
RandomForestRegressor, GradientBoostingClassifier,
GradientBoostingRegressor, HistGradientBoostingClassifier,
HistGradientBoostingRegressor, ExtraTreesClassifier,
ExtraTreesRegressor
```

3.3 SVM / KNN

- `SVC(kernel='rbf', probability=True)`，`LinearSVC()`
- `SVR(kernel='rbf')`，`LinearSVR()`
- `KNeighborsClassifier(n_neighbors=5)`，`KNeighborsRegressor(n_neighbors=5)`

```
from sklearn.svm import SVC, SVR, LinearSVC, LinearSVR
from sklearn.neighbors import KNeighborsClassifier,
KNeighborsRegressor
```

3.4 聚类 / 降维

- 聚类：`KMeans(n_clusters=k, n_init='auto', random_state=42)`，`DBSCAN(eps=0.5, min_samples=5)`
- 降维：`PCA(n_components=2)`，`TruncatedSVD(n_components=100)`，`FactorAnalysis()`

```
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA, TruncatedSVD,
FactorAnalysis
```

---

4) 评估指标（Metrics）

4.1 分类

- `accuracy_score(y, yhat)`
- `precision_score(y, yhat)`，`recall_score(y, yhat)`，`f1_score(y, yhat)`
- `roc_auc_score(y, proba)`，`average_precision_score(y, proba)`
- `classification_report(y, yhat)`
- 可视化：`ConfusionMatrixDisplay.from_estimator(...)`，`RocCurveDisplay.from_estimator(...)`，`PrecisionRecallDisplay.from_estimator(...)`

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score, average_precision_score,
classification_report, ConfusionMatrixDisplay, RocCurveDisplay,
PrecisionRecallDisplay
```

4.2 回归

- `r2_score(y, yhat)`

- `mean_absolute_error(y, yhat)`, `mean_squared_error(y, yhat, squared=False)` # RMSE
- `mean_absolute_percentage_error(y, yhat)`

```
from sklearn.metrics import r2_score, mean_absolute_error,
mean_squared_error, mean_absolute_percentage_error
```

4.3 无监督

- `silhouette_score(X, labels)`
- `calinski_harabasz_score(X, labels)`, `davies_bouldin_score(X, labels)`

```
from sklearn.metrics import silhouette_score,
calinski_harabasz_score, davies_bouldin_score
```

---

5) 模型解释 / 诊断

- 置换重要度：`permutation_importance(estimator, X, y, n_repeats=10, random_state=42)`
- 部分依赖：`PartialDependenceDisplay.from_estimator(estimator, X, features=[0, 'feature_name'])`
- 学习曲线：`learning_curve(estimator, X, y, cv=5, train_sizes=np.linspace(0.1,1.0,5))`
- 验证曲线：`validation_curve(estimator, X, y, param_name='model__C', param_range=[0.1,1,10], cv=5)`

```
from sklearn.inspection import permutation_importance,
PartialDependenceDisplay
from sklearn.model_selection import learning_curve,
validation_curve
```

---

6) 特征选择

- 过滤式：`SelectKBest(score_func=f_classif|mutual_info_classif, k=20)`
- 包装式：`RFE(estimator, n_features_to_select=20)`
- 嵌入式：`SelectFromModel(estimator, threshold='median')`

```
from sklearn.feature_selection import SelectKBest, f_classif, mutual_info_classif, RFE, SelectFromModel
```

7) 文本特征

- `CountVectorizer(ngram_range=(1,2), min_df=2)`
- `TfidfVectorizer(ngram_range=(1,2), min_df=2, max_features=200000)`
- 流水线组合：`make_pipeline(TfidfVectorizer(), LinearSVC())`

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
```

8) 持久化 / 工具

- 保存/加载：`joblib.dump(obj, 'model.joblib')`，`joblib.load('model.joblib')`
- 稀疏矩阵：`scipy.sparse.csr_matrix(...)`（与 `OneHotEncoder` 常搭配）
- 数据集：`fetch_california_housing`，`load_iris`，`load_breast_cancer`，`make_classification`，`make_regression`

```
import joblib
from sklearn.datasets import load_iris, load_breast_cancer, fetch_california_housing, make_classification, make_regression
```

9) 典型参数名（用于网格搜索）

- 线性：`LogisticRegression(C=..., penalty='l2', class_weight, solver='lbfgs'|'liblinear')`
- SVM：`SVC(C=..., kernel='rbf', gamma='scale', probability=True)`
- RF：`RandomForest* (n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features)`
- HGB：`HistGradientBoosting* (learning_rate, max_depth, max_leaf_nodes, min_samples_leaf)`
- KNN：`KNeighbors* (n_neighbors, weights='uniform'|'distance', p=1|2)`

示例：

```
param_grid = {
    "model__C": [0.1, 1, 10],
    "model__penalty": ["l2"],
    "model__solver": ["lbfgs", "liblinear"],
}
```

10) 最小工作流模板（分类）

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(X, y,
stratify=y, random_state=42)

pre = ColumnTransformer([
```

```
    ("num", StandardScaler(), X.select_dtypes('number').columns),
    ("cat", OneHotEncoder(handle_unknown='ignore'),
X.select_dtypes(exclude='number').columns),
])

pipe = Pipeline([
    ("pre", pre),
    ("model", LogisticRegression(max_iter=1000,
class_weight='balanced'))
])

param_grid = {"model__C":[0.1,1,10]}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="roc_auc",
n_jobs=-1)
grid.fit(X_train, y_train)
print(grid.best_params_)
print(classification_report(y_test, grid.predict(X_test)))
```